

Report

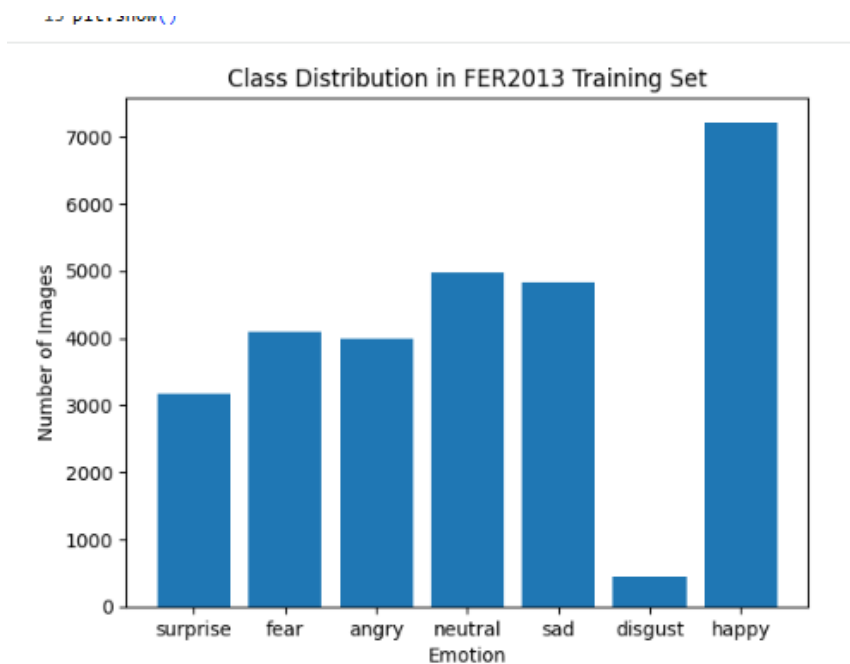
- **Problem Statement:**

Deep learning models rely heavily on the availability of large and well-balanced datasets. However, real-world datasets often suffer from class imbalance, where certain categories are significantly underrepresented. This imbalance leads to biased learning behavior, causing classifiers to favor majority classes while achieving poor recall and generalization on minority classes. In the Facial Expression dataset, the *disgust* emotion is severely underrepresented compared to other emotions, resulting in consistently low classification performance for this class. This project addresses this problem by using Generative Adversarial Networks (GANs) as a data augmentation strategy.

- **Description of Dataset & Imbalance Analysis:**

FER2013 facial expression dataset obtained from Kaggle was used for this project. It consists of grayscale facial images of size 48×48 pixels, categorized into seven emotion classes: angry, disgust, fear, happy, sad, surprise, and neutral. The dataset is provided with predefined training and testing splits.

An analysis of the class distribution reveals a severe imbalance among emotion categories. In particular, the disgust class contains significantly fewer samples compared to other emotions such as *happy* and *neutral*.



- **Details of GAN Architectures & Training:**

First, Vanilla GAN implemented a fully connected deep neural network for both the generator and discriminator. The generator takes a random noise vector and maps it to a synthetic 48×48 grayscale image. The discriminator receives either real or generated images and outputs a probability indicating whether the input image is real or fake. Binary cross-entropy loss was used for both networks.

Data augmentation techniques were applied to the original *disgust* images prior to GAN training to increase data diversity and stabilize learning. The best performance for the Vanilla GAN was achieved using a discriminator learning rate of 0.0005, a generator learning rate of 0.001, a batch size of 32, and 1500 training epochs.

DCGAN uses convolutional layers, which are more suitable for image generation tasks. The generator employs transposed convolution layers

combined with batch normalization and *ReLU* activations. The discriminator uses convolutional layers with *LeakyReLU* activations to distinguish real images from generated ones. DCGAN produced visually sharper images with improved structural consistency. The optimal DCGAN configuration used equal learning rates of 0.0002 for both the generator and discriminator, a batch size of 32, and 2500 training epochs. Similar to the Vanilla GAN, the DCGAN was trained only on the *disgust* class images.

- **Classifier Setup and Evaluation:**

The CNN classifier consists of multiple convolutional layers followed by pooling layers to extract spatial features from the 48×48 grayscale facial images. These layers are followed by fully connected layers and a *softmax* output layer corresponding to the seven emotion classes. *ReLU* activation functions were used in the convolutional layers to introduce non-linearity.

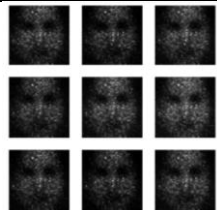
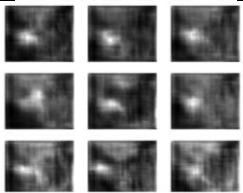
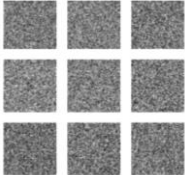
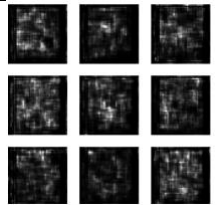
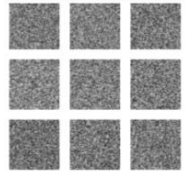
The classifier was trained using the sparse categorical cross-entropy loss function, Adam optimizer was used to update the network weights. Training was conducted on the original training split of the FER2013 dataset for the baseline model, and subsequently on an augmented dataset where GAN-generated disgust images were added to the training data to reduce class imbalance.

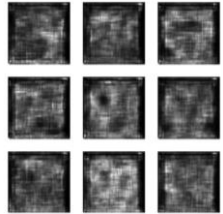
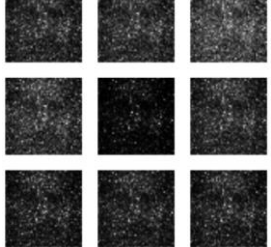
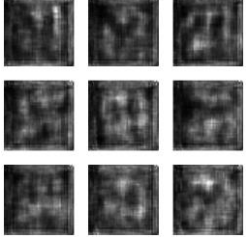
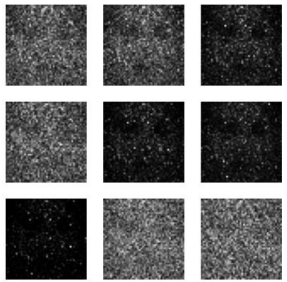
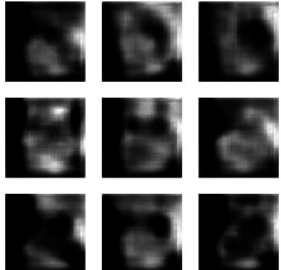
Model performance was evaluated on the predefined test split using multiple metrics, including overall accuracy, precision, recall, and F1-score. Due to the imbalanced nature of the dataset, particular emphasis was placed on the recall

and F1-score of the minority disgust class, as accuracy alone does not adequately reflect minority class performance. Confusion matrices were also used to analyze misclassification patterns across emotion classes.

• **Results & Comparisons:**

Multiple hyperparameter configurations were explored for both GAN architectures. The table below summarizes the most representative results.

GAN Type:	Learning Rate	Batch size	Epochs	Classification Report: Accuracy	Recall of minority class	Precision	F1-score	Generated images
Vanilla	0.002	64	2000	0.56	0.32	0.77	0.46	
DCGAN	0.002	64	3000	0.57	0.35	0.80	0.49	
Vanilla	D=0.0001 G=0.0001	32	300	0.56	0.41	0.70	0.52	
DC	D=0.002 G=0.0001	32	500	0.56	0.41	0.60	0.48	
Vanilla	D=0.001 G=0.0001	32	500	0.56	0.38	0.78	0.51	

DC	D=0.002 G=0.000 1	32	800	0.56	0.39	0.78	0.52	
Vanilla	D=0.000 5 G=0.001	32	1000	0.56	0.38	0.78	0.51	
DC	D=0.000 2 G=0.000 1	32	1500	0.56	0.39	0.78	0.52	
Vanilla	D = 0.0005 G = 0.001	32	1500	0.58	0.41	0.78	0.54	
DC	D=0.000 2 G=0.000 2	32	2500	0.57	0.41	0.88	0.56	

- ✓ Overall classification accuracy remained relatively stable across experiments (~56–58%).
- ✓ The recall of the minority disgust class improved consistently when GAN-generated images were introduced.

- ✓ DCGAN generally produced higher precision and F1-scores than Vanilla GAN.
- ✓ The best-performing model achieved a **disgust-class recall of 0.41** and an **F1-score of 0.56**, representing a notable improvement over the baseline.

Despite some visual blurriness in generated images, the classifier benefited from the increased sample diversity, indicating that perfect image quality is not strictly necessary for effective data augmentation.

- **Observations and Conclusion:**

This project demonstrates that GAN-based data augmentation is an effective strategy for mitigating class imbalance in facial expression recognition tasks. Both Vanilla GAN and DCGAN were able to generate meaningful synthetic samples for the underrepresented disgust class, leading to measurable improvements in recall and F1-score.

DCGAN consistently outperformed Vanilla GAN in terms of image quality and classification metrics, highlighting the importance of convolutional architectures for image synthesis.